# *Dynamic Product Pricing for Mercari Using Optimized Neural Networks*

Krishnil Chand
*Department of Computer Science & Information Systems*
*Fiji National University*
Suva, Fiji
krishnil.chand@fnu.ac.fj

Dr Anuraganand Sharma
*School of Information Technology, Engineering, Mathematics & Physics*
*University of the South Pacific*
Suva, Fiji
anuraganand.sharma@usp.ac.fj

***Abstract*—Determining the price of a product manually by a marketer is difficult and a risky task. Pricing the product too low will not enable a firm to cover its cost while pricing too high will lead to a loss of potential customers. This, in turn, will harm the sales and profit of the business. Pricing the product at the right price is of utmost importance for the firms to achieve their goals and many factors need to be taken into consideration. Mercari dataset mostly contains categorical and textual data which requires intensive data preprocessing. In this paper, we carried out comprehensive preprocessing on the Mercari dataset before presenting it to a dynamic pricing model using standalone Optimised Artificial Neural Network (ANN). The best hyperparameter is determined through Bayesian Optimization by way of Gaussian Processes and Gradient Boosted Decision Trees. The resultant hyperparameters are then trained on the ANN model with different optimizers such as Adam, RMSprop, Adagrad and Adamax. We achieved outstanding results by our model on the validation set that showed a 24% improvement on the winner of the Kaggle competition who uses an ensemble of 12 models.**

***Keywords—TF-IDF, RMSLE, One-Hot Encoding, Gaussian processes, Gradient Boosted Regression Trees, ensemble.***

## I. INTRODUCTION

Setting the price for a product to maximize profit is a difficult and complex task. This situation is even harder in the second-hand market, where marketers have no idea about the value of the product they are trying to sell to achieve buyer and seller satisfaction. For example, thousands of brands, seasonal pricing trend and fashion etc. should all be considered as an effective factor to set the price from a human perspective but what exact price should a seller tag on the product? It's highly challenging to predict the price of almost anything that is listed on online platforms and this is common with most online shopping platforms [1].

With the evolution of information and extensive research in the area of Artificial Intelligence (AI) over a few decades, industries are now exploring ways to automate different activities using Machine Learning algorithms and Deep Neural Networks. Most IT companies and start-ups have already taken a major jump in this field and have engaged their teams and resources for the research and development of AI applications. Today online shopping platforms are primarily driven by AI-powered algorithms [2].

Mercari is Japan's biggest community-powered shopping app that allows individuals to easily and safely buy and sell goods. When a user initially goes to the Mercari website, they see a picture of the item, price and number of likes on the item. To help their sellers price their products, Mercari had the challenge to build an algorithm that automatically suggests the right product prices to sellers on its app. Kaggle hosted the competition called the Mercari Price Suggestion from November 21, 2017, and run until February 21, 2018 [3].

This is an equilibrium challenge where the seller wants to sell at the best price to make a profit and the buyer wants to get the best price [4]. The tricky part is what the seller is willing to sell the item and what is the price that the buyer is willing to pay? The traditional way might be searching for the price of the related product, asking retailers' suggestion and make a discount from the original price. However, it will take a lot of effort, which might be more expensive compared to the product itself. Hence, a dynamic model is needed.

The rest of the paper is organized as follows. Section II provides a literature review of work done using the same dataset. Section III provides details of preprocessing techniques applied to the dataset. Section IV gives an overview of the Optimized Neural Network. Section V presents the experiments and results. Section VI concludes the paper with a discussion on futures research.

## II. RELATED WORKS

Das et. al [5] used Support Vector Machine Technique (SVM) and Back Propagation technique (BP) to predict the future price of a share in the stock market. A multilayer neural network was used to get the best price of a product and to maximise profit in a commercial firm during business expansion [6]. Past data in the form of the vector was given to the Artificial Neural Network (ANN). These inputs were Market Condition, Product Cost, Target Consumers and Miscellaneous Product Related Factors. The weights were initialised to some random values. The system adjusted its weights during the learning process. The model was able to give the optimum price for the given factors. Jayanram et al [7], used a three-layered back propagation Neural Network with *tanh* function to get the output with high accuracy. Six inputs were used namely, competitor price, discount, rating, delivery cost, Maximum Retail Price and product stock, across three companies (3 neurons at the hidden layer). Production cost was used as the bias. The outcome

showed that if the sellers would choose this model they would gain more profit. Neural Network with backpropagation has also been used by Chandrashekhar et al [8]. E-Commerce data was used to train and test the neural network consisting of five units at the input layer, three units at the hidden layer and one unit at the output layer. Production cost was used as the bias. The input used were product price, discount, items in stock, and sales. After 10,000 epoch the model was able to give desired results.

In the Mercari Price Suggestion Challenge [8], Kaggle provided Kernels with a specific setup and each competitor had to use the same setup. Both training a prediction had fit in a single Kernel which means ensembles had to be done in a single Kernel. The official benchmark model provided by Mercari has an RMSLE score of 0.82478. The gold winners used an ensemble of 12 models with 3 different datasets. The model was constructed using a multilayer Feedforward Neural network with sparse inputs [9]. They merged 12 resulting solutions using 1% of the dataset for validation (5% locally) and tuning the weights using a Lasso model. They trained 12 models with hidden size 256 and 64 and on about 200,000 features. ReLU activation function and Adam optimizer were used which had an RMSLE of 0.37665. The Silver Winner used the Ridge model and obtained an RMSLE of 0.38805. While Bronze Winner got an overall error rate of 0.39001. Another research conducted by Ali et al [10] conducted on the same dataset used the parameters: 500 components for the TruncatedSVD, two hidden layers for MLP with 100 neurons each, a learning rate of 0.001, regularization alpha value of 0.00001 and Adam. They received an RMSLE score of 0.5001.

## III. Data Preprocessing

Before the data can be used in the ML model, it needs to go through the different stages of data preprocessing. This section shows the comprehensive preprocessing of the Mercari dataset.

### *A. Data Analysis*

The Mercari's training dataset [3] holds 1,482,535 samples and has 8 columns as shown in Fig. 1.

The description of each attribute is as follows:

- train_id or test_id: a nominal integer data to label each record starting at zero.
- name: the title of the item given by the seller which is a text-based feature.
- item_condition_id: an ordinal feature describing the condition of the item given from new to very used (1, 2, 3, 4 or 5).
- category_name: a categorical feature that shows the category of the item.
- brand_name: a categorical feature indicating the brand of the item.
- shipping: a categorical feature showing whether the seller (1) or the buyer (0) pays for the shipment.
- item_description: a text-based feature that contains the description of the item.
- price: is the selling price of the item which is a numerical feature and is the class variable.

| train_id | name | item_condition_id | category_name | brand_name | price | shipping | item_description |
|---|---|---|---|---|---|---|---|
| 0 | MLB Cincinnati Reds T Shirt Size XL | 3 | Men/Tops/T-shirts | NaN | 10.0 | 1 | No description yet |
| 1 | Razer BlackWidow Chroma Keyboard | 3 | Electronics/Computers & Tablets/Components & P... | Razer | 52.0 | 0 | This keyboard is in great condition and works ... |
| 2 | AVA-VIV Blouse | 1 | Women/Tops & Blouses/Blouse | Target | 10.0 | 1 | Adorable top with a hint of lace and a key hol... |
| 3 | Leather Horse Statues | 1 | Home/Home Décor/Home Décor Accents | NaN | 35.0 | 1 | New with tags. Leather horses. Retail for [rm]... |
| 4 | 24K GOLD plated rose | 1 | Women/Jewelry/Necklaces | NaN | 44.0 | 0 | Complete with certificate of authenticity |
| ... | ... | ... | ... | ... | ... | ... | ... |

Fig. 1. Sample of Mercari Datasataset

The number of missing and unique values for each column is summarized in Table I. The number of rows with at least one missing value is 635,553 while the number of rows without missing values is 846,982.

It can be seen, in Fig. 2, that the class variable price has a right-skewed distribution. The mean is around 26.73 while the median is 170. The minimum price is as low as 0 and the maximum price is as high as 2009. Additionally, there are 874 samples with a price equal to 0.

Item Condition Id is originally encoded from good to bad using numerical values as 1 to 5. The number of categories in the "brand_name" column is 4810. Also, there are 632,682 missing values. For Shipping, there are 819,435 items for which buyers pay the shipping cost and 663,100 shipping cost paid by the seller. The category has 6,327 samples with missing values and 1287 unique values. It has been noted that each category name is made of multiple subcategories separated with "/". A sample of the category is shown in Fig. 3.

The category column has only 3, 4 or 5 subcategories. For example, the "Women/Tops & Blouses/Blouse" category is made of 3 subcategories which are "Women", "Tops & Blouses" and "Blouse". Table II gives details of each sub-category.

Due to the high number of unique descriptions, which is over a million, item description cannot be considered as a categorical feature. It will be kept as a text-based feature and text mining techniques will be applied to derive information. The same observation can be made to the name column similar to the item description.

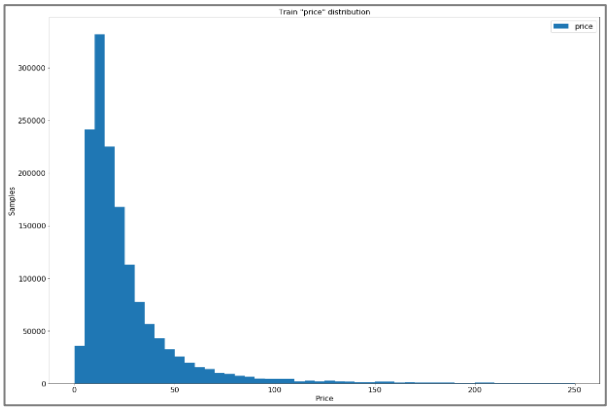


Fig. 2. Price Visualization

```
0                                          Men/Tops/T-shirts
1          Electronics/Computers & Tablets/Components & P...
2                                Women/Tops & Blouses/Blouse
3                      Home/Home Décor/Home Décor Accents
4                                 Women/Jewelry/Necklaces
                                ...
1482530                            Women/Dresses/Mid-Calf
1482531                          Kids/Girls 2T-5T/Dresses
1482532       Sports & Outdoors/Exercise/Fitness accessories
1482533                      Home/Home Décor/Home Décor Accents
1482534                       Women/Women's Accessories/Wallets
```

Fig. 4. Category Column

TABLE I. MISSING AND UNIQUE VALUES

| Feature | Missing Values | Unique Values |
|---|---|---|
| Train Id | 0 | 1,482,535 |
| Name | 0 | 1,225,273 |
| Item Condition Id | 0 | 5 |
| Category Name | 6,327 | 1,287 |
| Brand Name | 632,682 | 4,809 |
| Shipping | 0 | 2 |
| Item Description | 4 | 1,281,426 |
| Price | 0 | 828 |

TABLE II. SUBCATEGORIES

| Subcategories | Frequency | Null Values | Unique Values |
|---|---|---|---|
| Sub-Category 1 | 1476208 | 6,314 | 10 |
| Sub-Category 2 | 1476208 | 6,314 | 113 |
| Sub-Category 3 | 1476208 | 6,314 | 870 |
| Sub-Category 4 | 4389 | 1478146 | 7 |
| Sub-Category 5 | 3059 | 1479476 | 3 |

## B. *Feature Selection*

The first attempt of data cleaning is to normalize the data and remove free items i.e. those prices that has the value of 0. After performing this task, the dataset is reduced by 874 records.

The category column consists of many sub-categories and missing values. After splitting the categories, there are five categories as discussed in the previous section. Since there is a very huge number of nulls in sub-category 4 and 5, these will be dropped and the first 3 sub-categories will be used.

The brand name plays a significant part in the selling price. However, 632,682 of them are missing, and the reason might be that the items do not have a noticeable brand or the sellers forget the brand. To lower the dimension of the One-Hot Encoding, the top 300 brands are picked. The “NA” is filled by "missing brand", and the remaining will be replaced by “small brand", as they don't contribute much.

All data from the Item condition and Shipping column will be taken to the next phase of data preprocessing. Since both the item name and the item description describe what the item is. The simple combination of the item name and description will provide information about this item, and the NA’s in the description will be filled out accordingly by the item name.

## C. *Feature Transformation*

This section discusses the feature transformation techniques related to different kinds of data that were applied in this research namely; applying logarithmic scaling to the target variable, One-Hot Encoding on categorical features, and Natural Language Processing Techniques on textual data.

The evaluation metric in this competition is Root Mean Squared Logarithmic Error (RMSLE) to make errors on low price product more relevant than for higher prices. Therefore, log transformation is applied to price, to make this assumption available for model training as shown in Fig. 4.

The dataset contains categorical values for features; shipping, sub-categories and brand. It needs to be converted to numerical data since the model cannot work with categorical variables directly. To achieve this, One-Hot Encoding has been used which is a strategy in which each category value is converted into a new column and assigned a 1/0 (notation for true/false) value to the column indicating its presence or absence [11].

To apply text mining, we first combined the name and the description column and applied tokenization which is a fundamental technique for most Natural Language Processing (NLP) tasks. The descriptions and names were broken into sentences and then into tokens. The punctuations and the stop words were removed and the tokens were made to lower case. Also, only those words were selected which had a length of three characters.

N-Grams is one way to help machine learning to understand a word in its context to get a better understanding of the meaning of a word [12]. The general idea is to look at each pair (or triple, set of four, etc.) of words that occur next to each other. These co-occurring words are known as n-grams. We tried different n-grams and trigram worked the best.

Term Frequency-Inverse Document Frequency (TF-IDF) is used in machine learning and text mining as a weighting factor for features. This weight is a statistical measure used to assess how important a word is to a document in a collection or corpus. The importance increases proportionally with the number of times a word appears in the document but is offset by the frequency of the word in the corpus. TF-IDF can be used effectively for filtering stop-words in various subject fields including text summarization and classification [13].

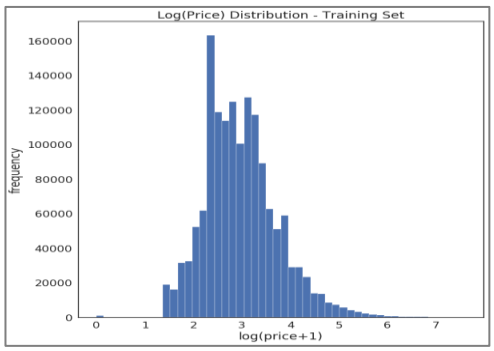


Fig. 3. Log Transformation of Price

The formulation of TF-IDF is as follows; TF is the term frequency *f* which is a measure of how frequently a term occurs in a document. This is given by (1) where *t* is the term (word) and d is the document (set of words).

$$tf(t,d) = \frac{f_{t,d}}{number\ of\ words\ in\ d} \quad (1)$$

IDF is the Inverse Document Frequency *idf* which defines the importance of a term. The formula is shown in (2) where df represents the occurrence of *t* in documents. 1 is added in the denominator to avoid dividing by 0.

$$idf(t) = log\left[\frac{n}{df(t)+1}\right] \quad (2)$$

Finally, TF-IDF is obtained by multiplying TF and IDF as shown in (3).

$$tfidf(t,d) = tf(t,d) * idf(t) \quad (3)$$

## IV. MODEL

We used a Multi-Layer Perceptron (MLP) neural networks [14] which is a group of perceptrons, organized in multiple layers, that can accurately answer complex questions. It utilizes backpropagation for training and uses a nonlinear activation function.

### A. Optimization Algorithm

Stochastic Gradient Descent (SGD) is a default standard optimization algorithm for gradient-based learning systems that calculates the derivative from each training data instance and calculates the update immediately [15] [16]. The Momentum is an acceleration method to help Stochastic gradient descent converge by reducing the oscillation. The gradient always follows the steepest direction, so that it's even better to let the gradient move along the direction of momentum [17].

The intuition behind adaptive learning rates is that it starts with big steps and finish with small steps. As the learning rate decays, smaller steps are taken to converge faster [18]. An adaptive learning rate can be observed in AdaGrad, AdaDelta, RMSprop and Adam optimizers.

### B. Neural Network Hyperparameters

A model parameter is internal to a learning network while a hyperparameter (shown in Table III) is an external parameter set by the operator of the neural network. Hyperparameters determine the structure of the neural network. Some of the methods of tuning hyperparameters for complex models are Grid search, random search, and Bayesian optimization.

TABLE III. COMMON HYPERPARAMETERS

| **ANN Structure Hyperparameters** | **Training Algorithm Hyperparameters** |
|---|---|
| Number of hidden layers: | Learning rate/ decay rate |
| Dropout: | Epoch, iterations and batch size |
| Activation function: | Optimizer algorithm |
| Weights initialization: | Momentum |

### C. Bayesian Optimizer

Bayesian Optimizer has gained popularity in recent years. It is an effective methodology that repeatedly trains the model with different hyperparameter values and attempts to observe the function shape produced by different parameter values. It then extends this function to predict the optimum values possible.

Gaussian processes (GPs) provide a principled, practical, probabilistic approach to learning in kernel machines. GPs have received increased attention in the machine-learning community over the past decade. Gradient Boosted Regression Trees (GBRT) are used to model a very costly function. The model is improved by evaluating the costly function sequentially on the next best level. Hereby determining the minimum function with the fewest possible evaluations [19]. GPs and GBRT have been applied in our experiment.

## V. EXPERIMENT AND DISCUSSION

In the preprocessing stage, those features that showed the importance of improving the efficiency of the model were selected. Except for train_id, all other attributes were considered. Those prices with the value of zero were removed. The category feature was split into sub-categories and the first three sub-categories were taken since they had significant observations. Top brands were taken into account and others were treated as small brands and no brands. Item condition and Shipping were used without any changes. Then One-Hot Encoding was applied on categorical data and TF-IDF was calculated on textual data. The Min-Max scaler was applied to normalize that data. The dataset was then split into 70% training and a 30% validation set which was then used is input in the model. The process and the range of features selected is shown in Fig. 5.

Moving on, the initial ANN model used has 2 hidden layers where the neurons at the first layer are the same as the number of features and the second hidden layer is half the size of the first hidden layer. The setup of the model with an initial set of hyperparameters is shown in Table IV. Table V shows the results obtained using the initial hyperparameters. Regularization of 0.0.01 was used for test case 10 and 11. Fig. 6 shows the fluctuations in RMSLE based on the number of features.

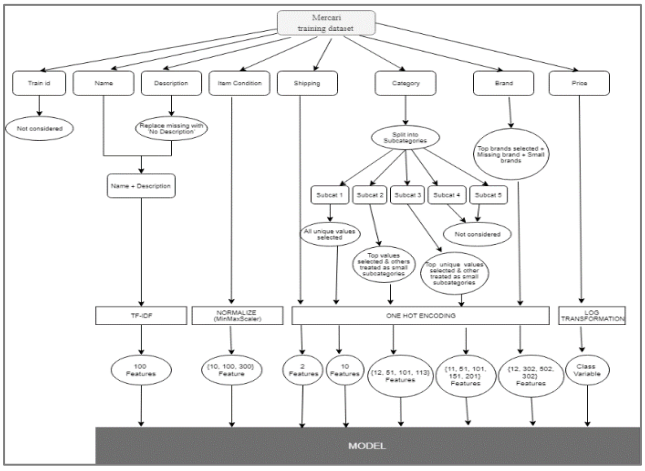


Fig. 5. Feature Selection & Transformation Process with Range of Features

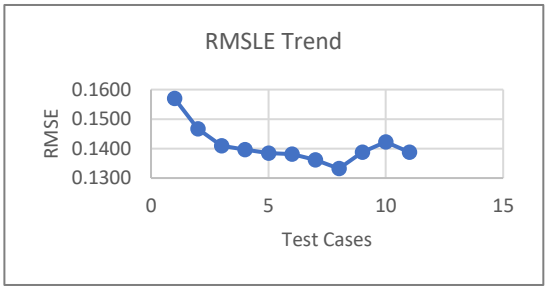


Fig. 6. Fluctuations in RMSLE according to the number of features

TABLE IV. RECOMMENDED HYPERPARAMETERS SETTINGS

| Parameters | Values |
|---|---|
| Epoch | 500 |
| Batch Size | 128 & 256 |
| Patience | 25, 50, 75, & 100 |
| Dropout | 0.2 |
| Optimizer | Adam |
| Loss Function | MSE |

TABLE V. FACTORS IMPACTING RMSLE

| Test Case | Subcat | Brand | Name & Descr | Total Feat. | Variance | MSE | RMSE | RMSLE |
|---|---|---|---|---|---|---|---|---|
| 1 | 32 | **12** | 0 | 47 | 0.2701 | 0.4883 | 0.6374 | 0.1570 |
| 2 | 32 | 302 | 0 | 337 | 0.3711 | 0.3500 | 0.5916 | 0.1467 |
| 3 | 112 | 302 | 0 | 417 | 0.4216 | 0.3218 | 0.5673 | 0.1410 |
| 4 | 212 | 302 | 0 | 517 | 0.4384 | 0.3128 | 0.5593 | 0.1397 |
| 5 | 324 | 302 | 0 | 629 | 0.4445 | 0.3089 | 0.5558 | 0.1385 |
| 6† | 324 | 502 | 0 | 829 | 0.4454 | 0.3084 | 0.5554 | 0.1381 |
| 7 | 324 | 502 | 10 | 839 | 0.4631 | 0.2988 | 0.5466 | 0.1362 |
| 8† | 324 | 502 | 100 | 929 | 0.4834 | 0.2873 | 0.536 | **0.1332** |
| 9 | 324 | 502 | 300 | 1129 | 0.4492 | 0.3068 | 0.5539 | 0.1388 |
| 10‡ | 324 | 502 | 100 | 929 | 0.4370 | 0.3186 | 0.5642 | 0.1423 |
| 11 | 324 | 502 | 300 | 1129 | 0.4492 | 0.3068 | 0.5539 | 0.1388 |

A various range of features was employed to check which one gives the best RMSLE. It has been observed that increasing the number of categorical and textual data helps by decreasing the RMSLE (see cases 6 and 8 with symbol †). Also, incorporating textual data into the model, as input, decreases the RMSLE even further. The more the categorical and textual data the better the RMSLE. However, this also impacts the variance. The variance increases as the number of features are increased. It can be seen that the variance decreases when regularization is employed (see case number 10 with symbol ‡).

### *A. Hyperparameter Optimization*

The Bayesian Optimizer with GPs and GBRT together with Adam optimizer for 10 Epoch and 12 iterations have been used for auto hyperparameter Optimization. The hyperparameters used in the Optimization algorithm are listed in Table VI. Mean Absolute Error (MAE) is used as a metric after each epoch. The Root Mean Squared Logarithmic Error (RMSLE) is employed as the loss function of the gradient descent.

The best result for validation loss is obtained with two hidden layers having 512 and 256 neurons respectively. Other information is shown in Table VII. The results released publicly by Kaggle is based on the validation dataset, therefore, this paper also uses a validation dataset to train and validate the model.

TABLE VI. HYPERPARAMETERS FOR AUTO HYPERPARAMETER TUNING

| Search Parameters | Range |
|---|---|
| Dense Layers | Integer(low=1, high=5) |
| Input Nodes | Integer(low=1, high=512) |
| Dese Nodes | Integer(low=1, high=512) |
| Activation Function | Categorical(categories=['relu', 'linear']) |
| Weight Initializers | Categorical(categories=['he_uniform', 'he_normal', 'glorot_uniform'] |
| Batch Size | Integer(low=1, high=256) |
| Regularization | Real(low=1e-4, high=0.5) |
| Dropout | Real(low=0.0, high=0.5) |
| Learning Rate | Real(low=1e-4, high=0.3, prior='log-uniform') |
| Weight Decay | Real(low=1e-6, high=1e-2) |
| Bias | Categorical(categories=['zeros', 'ones', 'random_normal']) |

TABLE VII. BEST HYPERPARAMETERS

| Search Parameters | Range |
|---|---|
| Activation Function | relu |
| Weight Initializers | he_uniform |
| Batch Size | 64 |
| Regularization | 0.0001 |
| Dropout | 0.000 |
| Learning Rate | 0.001 |
| Weight Decay | 0.001 |
| Bias | zeros |

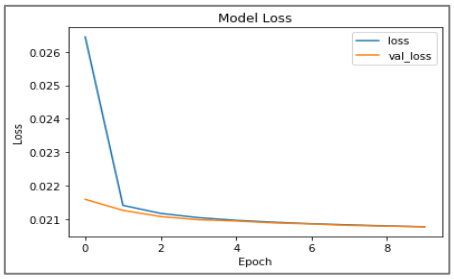


Fig. 7. Adam validation loss without TF-IDF

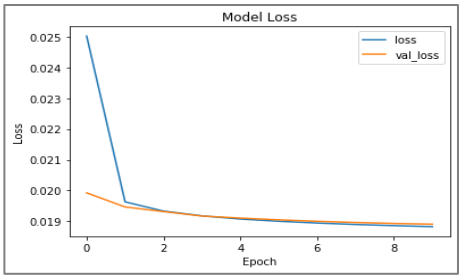


Fig. 8. Adam validation loss including TF-IDF

### *B. Result from Optimized ANN*

After getting a list of optimized hyperparameters, it is tested against Adam, RMSprop, and Adagrad and Adamax with 10 epochs. Firstly it is tested with features excluding TF-IDF. The number of features used is 579 (condition=1, shipping=2, subcategories=274, brand=302). The result is shown in Table VIII. Adam optimizer showed the best RMSLE of 0.14027. The validation loss graph is shown in Fig. 7.

Then textual data is included by the means of TF-IDF and using trigram with 100 maximum features. The number of features used is 679 (condition=1, shipping=2, subcategories=274, brand=302, TF-IDF=100). Adam optimizer showed the best RMSLE of 0.13305 (Table IX). The validation loss graph is shown in Fig. 8.

Comparing the results obtained including TF-IDF and without TF-IDF (Table X) shows that including textual data gives a better RMSLE. However, this increased the variance as well. Results without TF-IDF gives a better variance. Table XI shows the comparison between optimized and recommended hyperparameters. It is noticed that the model with the optimized hyperparameters perform better in terms of RMSLE and it is also able to fit well to the unseen data indicated by better variance value. Therefore, it can be concluded that optimized ANN performs better compared to ANN with recommended hyperparameter settings.

TABLE VIII. RESULTS WITHOUT TF-IDF

| Optimizer | Variance | MAE | MSE | RMSE | RMSLE |
|---|---|---|---|---|---|
| Adam | 0.42207 | **0.42728** | **0.32292** | **0.56826** | **0.14027** |
| RMSprop | 0.39834 | 0.43634 | 0.33750 | 0.58095 | 0.14292 |
| Adagrad | **0.04840** | 0.56353 | 0.53343 | 0.73036 | 0.17852 |
| Adamax | 0.42023 | 0.42823 | 0.32413 | 0.56933 | 0.14048 |

TABLE IX. RESULTS INCLUDING TF-IDF

| Optimizer | Variance | MAE | MSE | RMSE | RMSLE |
|---|---|---|---|---|---|
| Adam | 0.48009 | **0.40426** | **0.29011** | **0.53860** | **0.13305** |
| RMSprop | 0.45364 | 0.41482 | 0.30491 | 0.55219 | 0.13620 |
| Adagrad | **0.05913** | 0.55958 | 0.52747 | 0.72627 | 0.17755 |
| Adamax | 0.47292 | 0.40705 | 0.29480 | 0.5429 | 0.13389 |

TABLE X. RESULTS WITH AND WITHOUT TF-IDF

| | Results Including TF-IDF | Results Without TF-IDF |
|---|---|---|
| Variance | 0.05913 | **0.04840** |
| MAE | **0.40426** | 0.42728 |
| MSE | **0.29011** | 0.32292 |
| RMSE | **0.53860** | 0.56826 |
| RMSLE | **0.13305** | 0.14027 |

TABLE XI. OPTIMIZED VS RECOMMENDED HYPERPARAMETERS

| | Optimized Hyperparameters | Recommended Hyperparameters |
|---|---|---|
| Variance | **0.04840** | 0.48340 |
| MAE | 0.40426 | **0.40200** |
| MSE | 0.29011 | **0.28730** |
| RMSE | 0.53860 | **0.53600** |
| RMSLE | **0.13305** | 0.13320 |

### *C. Comparing Optimized ANN*

The solution obtained in this research shows a significant improvement in RMSLE by all the Optimization algorithms that were tested against. Results including TF-IDF with Adam optimizer gave a better RMSLE.

The result obtained using individual ANN is 0.13305 which is about 24% improvement on the validation set compared to the winner of the Kaggle competition where an ensemble of 12 models was used. Table XII shows the top 10 validation results publicly available on Kaggle.

TABLE XII. TOP 10 RANKED VALIDATION RESULTS FROM KAGGLE

| Rank | Team Name | RMSLE |
|---|---|---|
| 1 | Paweł and Konstantin | 0.37665 |
| 2 | Mercaring (Nima & Chahhou) | 0.38805 |
| 3 | 大王叫我来寻山 | 0.39001 |
| 4 | Bird | 0.39050 |
| 5 | Chenglong Chen | 0.39244 |
| 6 | John Wilson | 0.39245 |
| 7 | Anttip | 0.39382 |
| 8 | RDizzl3 and Sergei | 0.39446 |
| 9 | TetyanaYatsenko | 0.39446 |
| 10 | Basil | 0.39589 |

It can be observed that the number of neurons in the hidden layer plays a significant role in improving the result. The model in this research has 512 and 256 neurons at the first and second hidden layers respectively compared to 256 and 64 used by the winners and 100 neurons each by Ali et al. who scored an RMSLE of 0.5001 [10]. Gabriel used a single Deep Learning model i.e. CNN with GloVE for word embeddings on item name and item description. The model gave an RMSLE score of 0.41 which was at 35th place [20].

In the competition, the kernel was provided with a specific setup and the competitors were supposed to work with the given constraints. The only limitation in my research was the memory size which was 16GB which could not cater to many features when dealing with TF-IDF. As a result, a total of 679 features were used in the model compared to 200,000 features used by the gold winners, and 500 features used by Ali et al. TF-IDF was not incorporated by Ali et al. Alexandru used a Ridge Model with TF-IDF features to generate predictions. The model gave an RMSLE of 0.47088 [21].

Hyperparameter Optimization is critical in deciding the best hyperparameters for the model. It helps in selecting the right hyperparameter that is used for the machine learning algorithm. In this research, the best hyperparameters were observed by employing Bayesian Optimization through GPs and GBRT. Hyperparameter Optimization was not employed by any of the researchers discussed in this section and the literature review.

## VI. Conclusion and Future Directions

In this paper, the item price was determined dynamically using standalone optimized ANN together with extensive preprocess methods such as One-Hot-Encoding and TF-IDF. In the preprocessing stage, those features that showed correlations with the price were selected. Consequently, all attributes were considered, except for train_id. The category feature was split into sub-categories and the first three sub-categories were taken since they had significant observations. Top brands were considered and others were treated as small brands. Item condition and Shipping were used without any changes. TF-IDF was calculated on the textual data and One-Hot was applied on categorical data. Min-Max scaler was applied to normalize the data. The dataset was then split into 70% training and a 30% validation set before being fed into the model.

It has been observed that tuning the model using a hyperparameter Optimization algorithm improves the performance of the model significantly. The best hyperparameter was determined through Bayesian Optimization

by way of Gaussian Processes and Gradient Boosted Decision Trees. This led to the formation of the optimized ANN which was then tested using ANN model with different optimizers such as Adam, RMSprop, Adagrad, and Adamax. The best result achieved is RMSLE of 0.13305 which is a significant improvement compared to other researchers.

A key point to note is that individual ML models have comparatively higher RMSLE values compared to a model produced using Ensemble modelling. However, Ensemble-based approaches are becoming the current trend in machine learning competitions such as Kaggle, Netflix competition and KDD 2009. It helps in decreasing variance and improving predictions. Hence, the ensemble can be applied to further reduce the variance.

Furthermore, the TF-IDF was calculated for a maximum feature of 100. However, some researchers suggest that using a big corpus may improve the value of word vectors and ultimately the result of the prediction model. Therefore, more features can be engineered and tested with the model. Also, other vectorizations like DicVectorizer can be used on textual data to generate text features.

Finally, Some new hyperparameter optimization techniques that have shown promising results, such as XGBoost, can also be tried to see the improvement in the model. Additionally, some more complex models such as LSTMs and Convolutional Neural Nets can be tried. Regression models like FTRL and FM_FTRL can also be tried. The hyperparameters can be improved further by running more iterations, however, this is resource-intensive.


[1] J.-M. Chen and C.-I. Chang, "Dynamic pricing for new and remanufactured products in a closed-loop supply chain," *International Journal of Production Economics*, vol. 146, no. 1, pp. 153-160, 2013.

[2] X. Song, S. Yang, Z. Huang and T. Huang, "The Application of Artificial Intelligence in Electronic," in *Journal of Physics Conference Series*, 2019.

[3] "Kaggle," 2018. [Online]. Available: https://www.kaggle.com/c/mercari-price-suggestion-challenge/data. [Accessed 2019].

[4] S. J. Skripak, R. Parsons, A. Cortes and A. Walz, "Pricing Strategy," in *Fundamentals of Business*, Blacksburg, 2016, pp. 313 - 330.

[5] S. P. Das and S. Padhy, "Support Vector Machines for Prediction of Futures," *International Journal of Computer Applications*, vol. 41, no. 3, pp. 22-26, March 2012.

[6] M. Bilal, "A machine learning approach to pricing in expansive businesses," in *2017 International Conference on Computer Communication and Informatics (ICCCI)*, Coimbatore, 2017.

[7] K. M. N. Jayanram and J. M. S, "Dynamic Pricing in Ecommerce using Neural Network approach," *International Journal of Research*, vol. 3, no. 10, pp. 1272-1278, June 2016.

[8] A. M. Chranshekhar, H. P. Shivaraj and M. S. Jeevitesh, "Shot Term Price Forecasting of a Product in the field of E-Commerce using Artificial Neural Network approach," *International Journal of Engineering Research and Modern Education (IJERME)*, vol. 1, no. 2, pp. 17-25, 2016.

[9] S. Ueta, "Mercari Engineering Blog," December 2018. [Online]. Available: https://tech.mercari.com/entry/2018/12/05/183000. [Accessed 2020].

[10] A. A. S. Ali, H. Seker, S. Farnie and J. Elliott, "Extensive Data Exploration for Automatic Price Suggestion Using Item Description: Case Study for the Kaggle Mercari Challenge," in *2nd International Conference on Advances in Artificial Intelligence*, 2018.

[11] K. Potdar, C. Pai and T. Pardawala, "A Comparative Study of Categorical Variable Encoding Techniques for Neural Network Classifiers," *International Journal of Computer Applications*, vol. 175, no. 4, pp. 7-9, 2017.

[12] M. Pagliardini, P. Gupta and M. Jaggi, "Unsupervised Learning of Sentence Embeddings using Compositional n-Gram Features," in *Conference of the North American Chapter of the Association for Computational Linguistics*, New Orleans, 2018.

[13] A. Gaydhani, V. Doma, S. Kendre and L. Bhagwat, "Detecting Hate Speech and Offensive Language on Twitter using Machine Learning: An N-gram and TFIDF based Approach," in *IEEE International Advance Computing Conference 2018*, 2018.

[14] S. G. K. Patro and K. K. Sahu, "Normalization: A Preprocessing Stage," *IARJSET*, 2015.

[15] M. Cilimkovic, "Neural Networks and Back Propagation Algorithm," in *Cilimkovic*, 2010.

[16] L. Bottou, "Stochastic Gradient Descent Tricks," in *Neural Networks: Tricks of the Trade*, vol. 7700, Heidelberg, Springer, 2012, pp. 421-436.

[17] A. Sharma, "Guided Stochastic Gradient Descent Algorithm for inconsistent datasets," *Applied Soft Computing*, vol. 73, pp. 1068-1080, 2018.

[18] S. Ruder, "An overview of gradient descent optimization algorithms," *arXiv:1609.04747 [cs.LG]*, vol. 1, 2016.

[19] D. P. Kingma and J. L. Ba, "Adam: A Method for Stochastic Optimization," in *3rd International Conference for Learning Representations*, San Diego, 2015.

[20] C. Fan, D. Liu, R. Huang, Z. Chen and L. Deng, "PredRSA: a gradient boosted regression trees approach for predicting protein solvent accessibility," in *BMC Bioinformatics*, 2016.

[21] G. Moreira, "Mercari Price Prediction: CNN with GloVE (end-to-end single model)," 2018. [Online]. Available: https://www.kaggle.com/gspmoreira/cnn-glove-single-model-private-lb-0-41117-35th. [Accessed 2019].